Study Pack

Claude Certified Architect - Foundations | exam Tue 25 August 2026

How to use this pack. Follow the calendar in section 1. Each day: 10 minutes of recall drill, 50 minutes of new material from the domain notes, 45 minutes of practice questions from the separate questions booklet, then 15 minutes reviewing what you got wrong.

Read section 2 before anything else. It is worth more than any single fact.

The answer keys are not in this pack, on purpose. Mark your practice on screen.

Contents

CCAR-F - 15-Day Study Plan

Exam: Claude Certified Architect - Foundations 60 questions - 120 minutes - you pass at 720 out of 1000

Exam date: Tuesday 25 August 2026. Study period: Saturday 8 August to Monday 24 August.

> **Book the exam now** if you have not yet. Pearson VUE slots fill up, and you can cancel or move it free of charge up to 24 hours before. Booking early costs nothing and protects the date.

The daily shape - 2 hours

Use the same four blocks every day. The order matters: recall first, while you are fresh; practice last, when you are tired, because the exam is at the end of a long morning too.

Block	Time	What
1. Recall	10 min	Cheat sheet (one section) + Domain 4 facts. Cover the answers. Say them out loud
2. New material	50 min	Domain notes, or a course module
3. Practice	45 min	Drill questions, or `/cert-exam`
4. Review	15 min	Go through every wrong answer. Add one line to `LOG.md`

Block 4 is the one people skip. Do not. Reviewing a mistake teaches more than answering a new question correctly.

The calendar

Date	Day	Block 2 - new material	Block 3 - practice
Sat 8 Aug	[done]	Done - answer patterns	[done] Set 1: 11/20
Sun 9 Aug	-	Rest. Book the exam	-
Mon 10 Aug	1	Domain 1 notes 1.1-1.4 - start *Claude Code in Action*	`/cert-exam 15` - Domain 1
Tue 11 Aug	2	Domain 1 notes 1.5-1.7 - *Claude Code in Action* (Agent SDK, hooks)	**Domain 1 drill** (15)
Wed 12 Aug	3	Domain 2 notes - start *Intro to MCP*	`/cert-exam 15` - Domain 2
Thu 13 Aug	4	Finish *Intro to MCP*	**Domain 2 drill** (15)
Fri 14 Aug	5	Domain 3 notes - *Claude Code in Action* (commands, context)	`/cert-exam 15` - Domain 3
Sat 15 Aug	6	**Exercise 2** - configure a real repository (2 h, replaces blocks 2 and 3)	**Domain 3 drill** (15)
Sun 16 Aug	7	-	**Mock 1: `/cert-exam 60`, timed 120 min** + full review
Mon 17 Aug	8	Domain 4 notes 4.1-4.3	`/cert-exam 15` - Domain 4

Tue 18 Aug	9	Domain 4 notes 4.4-4.6 - *Building with the Claude API* (tool use)	**Retake Domain 1 drill**
Wed 19 Aug	10	*Building with the Claude API* (prompt engineering, agents)	**Domain 4 drill** (15)
Thu 20 Aug	11	Domain 5 notes - **Exercise 4**	`/cert-exam 15` - Domain 5
Fri 21 Aug	12	Re-read your two weakest domains	**Domain 5 drill** (15)
Sat 22 Aug	13	Skim the six scenarios (20 min, no memorising) - your two weakest domains	**Retake Domain 2 + 3 drills**
Sun 23 Aug	14	-	**Mock 2: Purcell's 60-question set, timed 120 min** + full review
Mon 24 Aug	15	Read `03-cheat-sheet.md` right through, twice	Redo only the questions you have ever got wrong
Tue 25 Aug	-	**EXAM**	-

What the extra time bought you

Compared with the 60-minute version:

- **Two timed mocks instead of one.** Mock 1 on 16 Aug is early enough to change the plan; Mock 2 on 23 Aug is the rehearsal.
- **Drill retakes at 7-day intervals.** Domain 1 on 11 and 18 Aug, Domains 2 and 3 on 13/15 and 22 Aug. Spaced repetition is what converts recognition into recall - and it is the method you already know from medical training.
- **The courses get real time**, instead of being squeezed between other work.
- **A `/cert-exam 15` on every domain** the day you study it, on top of my drills. That is roughly 170 practice questions in total.

How the week is shaped

Weekdays are 2 hours. **Sat 15, Sun 16 and Sun 23 need 2-3 hours** - Exercise 2 and the two mocks do not fit in less.

What was cut, and what was protected

The plan started at 20 days. It is now 15. Domain 3 lost a day, because you already scored 4 out of 6 there and it is memorisation you drill daily anyway. The scenarios day was merged into the weak-domain day, and two review days were removed.

Domain 4 kept three days. It is 20% of the exam and you scored 1 out of 5. Do not shorten it, even if you fall behind.

If you fall behind

Drop things in this order: 1. The hands-on exercises (except Exercise 2) 2. The courses 3. Domain 3 study - the cheat sheet covers it

Never drop: the daily facts, the drills, or the Sunday 23 mock.

Practice questions

Source	Questions	Multiple-response?
Mine - Set 1 + five domain drills	95	Yes
Community `/cert-exam` bank	77	No

Purcell's practice set (`1784098676646.pdf`)	60	Yes - 11 of them

Roughly 230 practice questions. Purcell's set is the closest thing to the real exam: written by someone who sat and passed it, blueprint-weighted 16/11/12/12/9, and pitched at the difficulty he met on the day. Save it for 23 August and take it cold - reading it early destroys your only realistic rehearsal.

Time on exam day

60 questions in 120 minutes = **2 minutes per question**.

But each scenario has a story you read once and then reuse. So the real plan is:

- **Skim** each scenario story, do not study it - about 1 minute each = 4 minutes
- About **1 minute 55 seconds** per question = 116 minutes

The questions are self-contained. Any detail you need is repeated inside the question, so reading the story carefully is wasted time. A candidate who passed finished in **90 of the 120 minutes**.

About 1 question in 5 asks for two answers. In a 60-question exam that is roughly 11 of them. Read the line that tells you how many to select - it is the easiest mark to throw away.

If a question takes more than 3 minutes, mark it and come back later. There is no penalty for guessing, so never leave a question empty.

Exam facts to remember

- You get **4 scenarios**, chosen at random from a set of **6**. All 6 are in the guide.
- Questions are multiple-choice or multiple-response. **Each question tells you how many answers to select.** Read that line.
- Score is 100-1000. You pass at **720**. You are measured against a fixed standard, not against other people.
- Your score report shows percent correct per domain. But pass or fail depends only on the **total** score.
- The credential lasts **12 months**. Renewal is free if you do not let it expire.
- If you fail: wait 14 days, then 30 days, then 90 days. Maximum 4 attempts in 12 months.

Files in this folder

```
prep/
 00-study-plan.md          <- this file
 01-answer-patterns.md     <- read first. Re-read on day 19
 02-course-map.md          <- which Partner Academy courses to take and skip
 notes/
   domain-1-agentic-architecture.md      27%
   domain-2-tool-design-mcp.md           18%
   domain-3-claude-code-config.md        20%
   domain-4-prompt-structured-output.md  20%  <- your weakest
   domain-5-context-reliability.md       15%
   scenarios.md                          all 6 exam scenarios
 practice/
   set-01-questions.md       20 questions (day 1)
   set-01-answers.md         answers and explanations
   drill-domain-1.md         15, questions and answers in one file (days 2-4)
   drill-domain-2.md         15 (days 5-6)
   drill-domain-3.md         15 (days 7-9)
   drill-domain-4.md         to be written (days 11-13)
   drill-domain-5.md         to be written (days 14-15)
   daily-facts-domain-4.md   5 minutes, EVERY DAY from day 2
   set-02-*.md               30, mixed  -  written after your day-1 results
```

```
  set-03-*.md               60, full test  -  written after your day-10 results
exercises/
  README.md                 the guide's 4 exercises, with checkpoints
```

Note on the two file types: the drills keep questions and answers in one file, so cover the answers while you work. The numbered sets keep them in two files, so you cannot see the answers during a timed test.

A note about language

The notes and explanations are written in simple English.

The **practice questions are not**. They use the same long, formal English as the real exam. This is on purpose - reading that style quickly is part of the test.

If a practice question is hard to understand, ask about it by number. Working out what a difficult question is really asking is useful practice for exam day.

How the Exam Builds Its Questions

Read this first. Re-read it on day 19.

This file is not about facts. It is about **how the questions are written**. If you understand the pattern, you can remove wrong answers even when you are unsure of the right one.

All 12 sample questions in the official guide follow the same pattern.

Part 1 - The main rule

For every question, ask two things, in this order.

Question 1: What is the real cause of the problem?

The question text almost always tells you. Look for it.

Example from the guide (Sample Q7): "the coordinator's logs show it split the topic into three subtasks." The text names the coordinator. So every answer that blames a different agent is wrong.

Question 2: What is the smallest change that fixes that cause?

Not the biggest change. Not the most advanced one. The smallest change that solves the problem the text describes.

If two answers both fix the real cause, choose the simpler one.

One exception: if the question says the fix must be guaranteed, choose the strong method instead. See Part 2.

Part 2 - The strength ladder

Some fixes are weak but cheap. Some are strong but expensive. The exam wants the **weakest fix that still meets the requirement**.

Here is the ladder, from weakest to strongest:

Level	Fix	Choose it when
1	Write better **tool descriptions**	The model picks the wrong tool, and the descriptions are short or too similar
2	Add **few-shot examples** (2-4 examples in the prompt)	Behaviour is inconsistent in unclear cases
3	Write **explicit criteria** in the prompt	Too many false alarms; the current instruction is vague
4	Use **programmatic enforcement** (hooks, prerequisite gates, forced `tool_choice`)	The rule must always work
5	Build a **separate classifier or ML system**	Almost never the right answer on this exam

"Programmatic enforcement" means: code checks the rule, not the model. The model cannot skip it.

"Prerequisite gate" means: block step B until step A has finished.

When does level 4 win?

Level 4 beats levels 1-3 when the question mentions any of these:

- Money (refunds, payments, billing)
- Identity checks
- Policy limits or thresholds

- The words **deterministic**, **guaranteed**, **must**, or **never**

Reason from the guide: prompt instructions have a "non-zero failure rate." That means they fail sometimes. Sometimes is not acceptable when money is involved.

Examples: - Sample Q1 - the agent must verify the customer before a refund. Money. -> Level 4. - Sample Q2 - the agent picks the wrong tool; descriptions are short. No money. -> Level 1. - Sample Q3 - the agent escalates the wrong cases. No money. -> Level 2 and 3.

Part 3 - The five kinds of wrong answer

Every wrong answer in the guide's samples is one of these five kinds. Learn to see them.

Kind 1 - Too big a solution

It builds new systems before trying simple fixes.

Words that signal this: "train a classifier", "deploy a separate model", "add a routing layer", "build a service".

Examples from the guide: - Q2-C: a routing layer that reads user input before every turn. The guide calls this "over-engineered." - Q3-C: a classifier trained on old tickets. The guide says this "requires labeled data and ML infrastructure when prompt optimization hasn't been tried."

Kind 2 - It blames a part that works

The question tells you which part is broken. Any answer pointing at a different part is wrong.

Example: Q7-A, Q7-C and Q7-D all blame agents that the guide says were "working correctly within their assigned scope."

Kind 3 - The feature does not exist

The answer sounds confident but describes a flag or file that is not real.

Not real: - `--batch` - `CLAUDE_HEADLESS=true` - `.claude/config.json` with a `commands` list

Real: `-p` / `--print` - `--output-format json` - `--json-schema` - `.claude/commands/` - `.claude/skills/` - `.claude/rules/` - `.mcp.json` - `~/.claude.json` - `--resume` - `fork_session` - `/memory` - `/compact`

If you memorise the real list, these questions become easy.

Kind 4 - It solves a different problem

The answer is reasonable, but it fixes something the question did not ask about.

Examples: - Q1-D: turns tools on and off. But the problem was tool **order**, not tool **availability**. - Q3-D: uses sentiment analysis. But the problem was case **difficulty**, and the guide says "sentiment doesn't correlate with case complexity."

Kind 5 - It moves the work to people, or makes things worse

Examples: - Q12-B: ask developers to split large pull requests. The guide says this "shifts burden to developers without improving the system." - Q12-D: run three reviews and report only issues found in two of them. The guide says this "would actually suppress detection of real bugs."

Part 4 - Rules that are always true

Never the right answer

- **Self-reported confidence** used to decide when to escalate. The model is already confident about the hard cases it gets wrong.
- **Sentiment analysis** used to measure how difficult a case is.
- **A bigger context window** used to fix inconsistent quality. Context size does not fix attention.
- **Voting across several runs** to filter results. This hides real bugs.
- **Reading the model's text** to decide when the loop should stop. Use `stop_reason` instead.

- **A fixed maximum number of loops** as the main way to stop. (As a safety limit it is fine.)
- **Hiding errors** by returning empty results and calling them a success.
- **Stopping the whole workflow** because one subagent failed.
- **Guessing** when a tool returns several matches. Ask for another identifier instead.

Usually the right answer

- Error responses that include a category and a retry flag, instead of one generic message.
- Passing context to a subagent **in its prompt**. Subagents inherit nothing automatically.
- Splitting a large review into one pass per file, **plus** one extra pass across files.
- Using a second, separate Claude instance to review, instead of asking the same one to check itself.
- Making schema fields optional so the model does not invent values.
- Sending all subagent messages through the coordinator.
- Escalating at once when the customer asks for a human.

Part 5 - Word signals in the question

When you see the words on the left, think of the answer on the right.

Words in the question	What to choose
"most effective **first step**"	The cheap, simple fix (level 1-3), not the big one
"deterministic", "guaranteed", "must never"	Programmatic enforcement (level 4)
"the logs show the coordinator..."	The cause is named - fix that part
"minimal descriptions"	Improve the tool descriptions
"blocking", "developers wait"	Real-time API, not batch
"overnight", "next morning", "weekly"	Batch API is fine here
"inconsistent results", "contradictory feedback"	Split into smaller, focused passes
"false positives", "developer trust"	Write explicit criteria
"hangs", "waiting for input"	`-p` / `--print`
"every developer when they clone the repo"	Project scope: `.claude/...` in the repo
"spread throughout the codebase"	`.claude/rules/` with glob patterns
"dozens of files", "architectural decisions"	Plan mode
"single-file fix with a clear stack trace"	Direct execution

Part 6 - Questions with more than one answer

The exam tells you how many answers to select. Read that line.

Two useful patterns:

1. When it says "select two," the two answers are usually **two different layers** of a solution. They are not two versions of the same idea. Example: change the schema **and** change the prompt.
2. One answer is often obviously correct. The second correct answer is often the one that handles the **edge case** the first one misses.

Part 7 - Your 30-second method for each question

1. Read the **last sentence** of the question first. That is what is actually being asked.
2. Look for the word signals in Part 5.
3. Remove any answer that describes a feature that does not exist.
4. Remove any answer that blames a part the question says is working.
5. From what is left, choose the answer that fixes the named cause using the **weakest method that still meets the requirement**.

Domain 1 - Agentic Architecture & Orchestration (27%)

This is the biggest domain. About 16 of the 60 questions come from here.

It has 7 task statements. They are all about the same thing: **how work is split up, given to other agents, controlled, and continued later.**

1.1 The agentic loop

An **agentic loop** is the cycle where Claude asks for a tool, you run it, and you send the result back.

Only one thing controls the loop: **`stop_reason`**.

```
send request -> look at stop_reason
 |-- "tool_use"  -> run the tools Claude asked for
 |                  -> add Claude's message (with the tool_use blocks) to the history
 |                  -> add the tool results as a user message
 |                  -> repeat
 \-- "end_turn"  -> stop. Show the final answer.
```

`stop_reason` values

The exam tests two: `"tool_use"` and `"end_turn"`.

The real API has six: `end_turn`, `max_tokens`, `stop_sequence`, `tool_use`, `pause_turn`, `refusal`. Know all six. Answer with the two from the guide.

Rules for tool results

- Add tool results to the conversation history. This lets the model use the new information in the next step.
- Every `tool_use` block needs one matching `tool_result`, with the same `tool_use_id`.
- Send **all** results from one Claude message back together, in **one** user message.

Three wrong ways to stop the loop

The exam will offer these as answers. They are always wrong.

1. Reading Claude's text to see if it says something like "I am finished."
2. Using a maximum number of loops as the **main** way to stop. (Using it as a safety limit is fine. Using it as the mechanism is not.)
3. Checking if the response contains text. Text and tool requests often appear together.

Model-driven vs pre-configured

- **Model-driven:** Claude decides which tool to call next, based on the situation.
- **Pre-configured:** you write a fixed decision tree or a fixed order of tools.

Choose model-driven - **unless** section 1.4 applies (when a rule must always work).

1.2 Coordinator and subagent design

A **coordinator** is the main agent. A **subagent** is a helper agent that does one job.

Hub-and-spoke

All messages go through the coordinator. Subagents never talk to each other.

Why: you get better visibility, one place for error handling, and control over what information moves where.

Subagents start empty

A subagent does **not** receive the coordinator's conversation history. It does not share memory between calls. Anything it needs must be written in its prompt.

What the coordinator does

- Splits the task into parts (**task decomposition**)
- Gives parts to subagents
- Collects the results
- Decides **which** subagents to call, based on how complex the request is

That last point matters. A good coordinator does not always run every subagent. It chooses.

The main failure to remember

Task decomposition that is too narrow.

This is Sample Q7 in the guide. The topic was "impact of AI on creative industries." The coordinator split it into three subtasks, and all three were about visual art. Music, writing and film were never researched.

Every subagent did its job correctly. The **split** was wrong.

Exam rule: if the question says coverage is incomplete but each agent worked correctly, the answer is the coordinator's task decomposition.

Two more coordinator skills

- **Scope partitioning:** give each subagent a different subtopic, or a different type of source. This stops two agents doing the same work.
- **Iterative refinement loop:** the coordinator reads the synthesis output, looks for gaps, sends new targeted questions to the search and analysis subagents, then runs synthesis again. It repeats until coverage is good enough.

1.3 Starting subagents and passing context

- Subagents are started with the **Task tool**.
- The coordinator's `allowedTools` **must include** `"Task"`. Without it, the coordinator cannot start any subagent. This is a common exam question.
- **AgentDefinition** is the configuration for each subagent type. It holds the description, the system prompt, and which tools that subagent may use.
- **To run subagents in parallel:** send several Task tool calls **in one single response**. If you send them in separate turns, they run one after another.
- Put the earlier findings **directly in the subagent's prompt**. For example, give the synthesis subagent the actual web search results and document analysis output.
- Use a **structured format that separates content from metadata** (source URLs, document names, page numbers). This keeps the source information attached to each finding.
- Write coordinator prompts that describe **goals and quality standards**, not step-by-step instructions. Step-by-step prompts stop subagents from adapting.
- `fork_session` creates separate branches from one shared analysis. Use it to compare different approaches.

1.4 Making rules that always work

This is the most important idea in Domain 1.

Programmatic enforcement	Prompt-based guidance
Hooks, prerequisite gates	System prompt text, few-shot examples

Always works	Fails sometimes
Use for: identity checks before money operations, policy limits, compliance	Use for: style, preferences, judgement

The guide's exact reasoning: prompt instructions have a "non-zero failure rate." That means they fail sometimes. That is not acceptable when money is involved.

The standard example: block `process_refund` until `get_customer` has returned a verified customer ID.

If the question involves money, any answer using prompts or examples is wrong.

Requests with several problems in one message

Correct handling: 1. Split the message into separate items. 2. Investigate each one **in parallel**, using shared context. 3. Combine the results into **one** answer for the customer.

Wrong: handle them one at a time, or ask the customer to choose which one to fix first.

Handoff to a human

The human agent **cannot see the conversation**. So the handoff summary must include: - Customer ID - Root cause analysis - Refund amount - Recommended action

---

1.5 Hooks

A **hook** is code that runs automatically at a certain point. The model cannot skip it.

Two types are tested:

`PostToolUse` - runs after a tool returns, before the model sees the result. Main use: **normalise different data formats**. Example from the guide: one service returns Unix timestamps, another returns ISO 8601, another returns a numeric status code. The hook converts them all to one format first.

Tool call interception - runs before a tool is called, and can block it. Main use: **stop actions that break a policy**. Example from the guide: block refunds above $500 and send them to human escalation instead.

Choose a hook whenever a business rule must always be followed.

1.6 How to split up complex work

Pattern	Use it when	Example
Prompt chaining (fixed steps, always the same)	The work is predictable	Review each file, then run one extra pass across all files
Dynamic decomposition (steps depend on findings)	The work is open-ended	"Add tests to a legacy codebase": first map the structure, then find the risky areas, then build a plan that changes as you discover dependencies

Large code reviews

The correct pattern is: **one pass per file, plus one separate pass across files.**

This fixes **attention dilution** - when a model reviews too much at once, quality drops and it gives inconsistent answers.

A bigger context window does not fix this. The guide is clear: larger context does not improve attention quality. Any answer suggesting a bigger model or bigger window is wrong.

1.7 Sessions: continue, branch, or restart

- **`--resume <session-name>`** continues one specific earlier conversation.

- `fork_session` creates separate branches from one shared starting point. Example: compare two refactoring approaches using the same codebase analysis.
- **Tell a resumed session what changed.** If files were edited since last time, say which files. Then the agent re-reads only those, instead of exploring everything again.

Resume or start fresh?

Situation	Do this
Earlier context is mostly still correct	Resume
Earlier tool results are **stale** (out of date)	Start a **new** session and give it a written summary

The guide states this directly: starting fresh with a structured summary is more reliable than resuming with stale tool results.

Quick review list for Domain 1

1. The loop is controlled by `stop_reason`. `tool_use` = continue. `end_turn` = stop.
2. `allowedTools` must include `"Task"` to start subagents.
3. Subagents receive nothing automatically. Put context in the prompt.
4. Parallel = several Task calls in **one** response.
5. Coverage is incomplete but every agent worked = the coordinator's split was too narrow.
6. Money, identity, or policy limits = hooks or gates. Never prompts.
7. `PostToolUse` changes results. Interception hooks block calls.
8. Big review = one pass per file + one pass across files.
9. Stale tool results = new session with a summary, not `--resume`.

Domain 2 - Tool Design & MCP Integration (18%)

About 11 of the 60 questions come from here. There are 5 task statements.

Two ideas run through the whole domain:

1. **Tool descriptions are how the model chooses a tool.** Bad descriptions cause wrong choices.
2. **Errors must contain enough detail for the agent to decide what to do next.**

2.1 Designing tool interfaces

The most important fact in this domain

Tool descriptions are the main way the model decides which tool to use. If a description is short, the model cannot tell similar tools apart.

What a good description contains

- What the tool does
- **Which input formats** it accepts
- **Example queries**
- **Edge cases**
- **When to use it instead of a similar tool**

The failure to remember

Descriptions that are unclear or overlap cause **misrouting** (the model picks the wrong tool).

The guide's example: `analyze_content` and `analyze_document` have almost the same description.

Three fixes, from cheapest to most expensive

Make the descriptions longer and clearer. This is the answer when the question asks for the "most effective first step" (Sample Q2).

Rename the tool and rewrite its description, so the two no longer overlap. Guide example: rename `analyze_content` to `extract_web_results`, with a web-specific description.

Split one general tool into several specific tools, each with a defined input and output. Guide example: split `analyze_document` into `extract_data_points`, `summarize_content`, and `verify_claim_against_source`.

Also check the system prompt

Words in the system prompt can create unwanted links to tools. This can override good descriptions. The guide calls this "keyword-sensitive instructions."

Example: if the system prompt says "your main purpose is to produce reports," the model may send every request containing the word "report" to `generate_report`.

One thing to be careful about

Combining several tools into one general tool (for example, one `lookup_entity` that accepts any ID) is a **valid design choice**. But it is the wrong answer when the question asks for a "first step." It costs more effort than fixing the descriptions.

2.2 Structured error responses for MCP tools

MCP uses an **`isError` flag** to tell the agent that a tool failed.

The four error categories

Category	Example	What the agent should do
Transient	Timeout, service is down	Retry
Validation	Bad input	Fix the input, then retry
Business	The action breaks a policy	Explain it to the user. Do not retry
Permission	Not allowed	Escalate or explain

Why one generic error is wrong

A message like `"Operation failed"` gives the agent nothing. It cannot decide whether to retry, explain, or escalate.

On the exam, any answer offering a single generic error message is wrong.

What to return instead

```
{
 "isError": true,
 "errorCategory": "transient",
 "isRetryable": true,
 "message": "A clear sentence a person can read"
}
```

For a business rule violation, include `retriable: false` **and a customer-friendly explanation**, so the agent can explain the policy to the user.

Local recovery in subagents

A subagent should fix transient failures itself. It only reports failures it **cannot** fix. When it reports one, it includes **partial results and what it tried**.

Access failure is not the same as an empty result

- **Access failure:** the tool could not reach the service. The agent may need to retry.
- **Valid empty result:** the search worked, and found nothing.

Treating these as the same thing is a wrong answer on the exam.

2.3 Which agent gets which tools, and `tool_choice`

Too many tools is a real problem

The guide gives a number: **18 tools instead of 4-5** makes tool selection unreliable. More tools means a harder decision.

Also: an agent with tools outside its job will misuse them. Example: a synthesis agent that starts doing web searches.

Scoped cross-role tools

Sometimes an agent needs one small ability from another role. Give it a **narrow tool**, not the full set.

Sample Q9 in the guide: the synthesis agent needs to check facts. 85% of checks are simple (dates, names, numbers). Give it a small `verify_fact` tool for those. The other 15% still go through the coordinator.

This is **least privilege** applied to tools: give each agent only what it needs.

Replace general tools with limited ones

Guide example: replace `fetch_url` (which can fetch anything) with `load_document` (which validates that the URL is a document).

`tool_choice` values

Value	What happens
`{"type": "auto"}`	The model decides. It **may return text instead of calling a tool**
`{"type": "any"}`	The model **must** call a tool, but chooses which one
`{"type": "tool", "name": "..."}`	The model must call **that specific** tool
`{"type": "none"}`	The model cannot use tools. *Real, but the guide does not test it*

Short version: **auto may talk - any must act - forced picks the actor.**

How the exam uses them: - `"any"` - you have several extraction schemas and do not know the document type yet. Or the model keeps replying with text instead of extracting. - Forced - you need one specific tool to run first. Example: run `extract_metadata` before any enrichment tool. Later steps happen in the next turns.

2.4 Setting up MCP servers

Where configuration files go

Scope	File	Use it for
Project	`.mcp.json` (in the repository)	Shared team tools. Version-controlled
User	`~/.claude.json`	Personal or experimental servers

Learn this table. It appears often.

Keeping secrets out of the repository

Use **environment variable expansion** in `.mcp.json`, like `${GITHUB_TOKEN}`. The file can be committed. The secret is not in it.

All servers work at the same time

Tools from every configured server are discovered when the connection opens. Project servers and personal servers are available together.

Tools vs resources

- **Tools** = actions the agent can take.
- **Resources** = a way to show the agent **content catalogues**: lists of issues, documentation structures, database schemas.

Resources let the agent see what data exists **without making exploratory tool calls** first.

Two judgement questions the exam likes

1. **Improve MCP tool descriptions** so the agent understands what they can do. Otherwise it prefers built-in tools like Grep, even when the MCP tool is better.
2. **Use an existing community MCP server** for standard integrations such as Jira. Build a custom server only for workflows specific to your team.

2.5 The built-in tools

Tool	What it does	Choose it when
Grep	Searches **inside** files	Finding function callers, error messages, import statements

Glob	Matches **file paths**	Finding files by name or extension, such as `**/*.test.tsx`
Read	Loads a whole file	Following imports, tracing how code works
Write	Writes a whole file	When Edit cannot find unique text
Edit	Changes part of a file, by matching unique text	Small, precise changes
Bash	Runs shell commands	-

Remember: Grep looks inside files. Glob looks at file names and paths. This pair is confused often.

When Edit fails

If Edit cannot find unique text to match, use **Read + Write** instead. This is a direct exam fact.

How to explore a codebase

Do not read every file first. Instead: 1. Use **Grep** to find entry points. 2. Use **Read** to follow imports and trace the flow. 3. Repeat.

Tracing a function through wrapper modules

First find all the exported names. Then search for each name across the codebase.

Quick review list for Domain 2

1. Tool descriptions decide tool choice. Improve them before anything else.
2. A description needs: inputs, examples, edge cases, and when to use it instead of a similar tool.
3. Errors carry `errorCategory` + `isRetryable` + a readable message.
4. Four categories: transient, validation, business, permission.
5. An access failure is not an empty result.
6. 18 tools is bad. 4-5 is good. Add small cross-role tools for the common case.
7. `auto` may return text - `any` must call something - forced calls a named tool.
8. `.mcp.json` = project and shared. `~/.claude.json` = personal.
9. `${VAR}` keeps secrets out of the repository.
10. MCP **resources** are content catalogues. They reduce exploratory calls.
11. Grep = inside files. Glob = file paths. Edit fails on unclear matches -> Read + Write.

Domain 3 - Claude Code Configuration & Workflows (20%)

About 12 of the 60 questions come from here. There are 6 task statements.

This domain is mostly **memorisation**: file paths, frontmatter keys, and command-line flags. That makes it the easiest domain to improve quickly. If you learn the tables in this file, you will get these questions right.

3.1 The CLAUDE.md hierarchy

CLAUDE.md is a file with instructions that Claude Code loads automatically.

There are three levels:

Level	Path	Shared with your team?
User	`~/.claude/CLAUDE.md`	[no] **No.** Only for you
Project	`.claude/CLAUDE.md` or `CLAUDE.md` in the repository root	[done] Yes
Directory	`CLAUDE.md` inside a subfolder	[done] Yes, but only for that folder

The standard exam question

A new team member does not get the team's instructions. Everyone else does.

Cause: the instructions are in a **user-level** file (`~/.claude/CLAUDE.md`). This file is not shared through version control.

Fix: move them to the project level.

`@import`

`@import` lets one CLAUDE.md file reference another file. This keeps files small and avoids copying the same text into several places.

Example: each package has its own CLAUDE.md, and imports only the standards documents that apply to it. The package maintainer chooses which ones.

`.claude/rules/`

This folder holds separate rule files by topic, instead of one very large CLAUDE.md.

Example files: `testing.md`, `api-conventions.md`, `deployment.md`.

`/memory`

This command shows which memory files are currently loaded.

Use it when Claude Code behaves differently in different sessions.

3.2 Slash commands and skills

Type	Project scope (shared)	User scope (personal)
Slash commands	`.claude/commands/`	`~/.claude/commands/`
Skills	`.claude/skills/`	`~/.claude/skills/`

Sample Q4 in the guide

You want a `/review` command that every developer gets when they clone the repository.

Answer: `.claude/commands/` inside the repository.

The wrong answers: - `~/.claude/commands/` - personal, not shared - CLAUDE.md - holds context, not command definitions - `.claude/config.json` with a `commands` list - **this does not exist**

SKILL.md frontmatter

A skill is a folder containing a `SKILL.md` file. The top of that file has settings called **frontmatter**.

Three keys are tested:

Key	What it does	Use it when
`context: fork`	Runs the skill in a **separate context**, away from the main conversation	The skill produces a lot of output (codebase analysis) or exploratory output (brainstorming) that would fill up the main conversation
`allowed-tools`	Limits which tools the skill may use	You want to prevent destructive actions. Example: allow only file writing
`argument-hint`	Asks the developer for a missing parameter	The skill needs a parameter and was called without one

Important: `paths:` is **not** a SKILL.md key. It belongs to `.claude/rules/`. Mixing these two sets of keys is the most common mistake in this domain.

Personal versions of a shared skill

Create your version in `~/.claude/skills/` **with a different name**. This way your teammates are not affected.

Skill or CLAUDE.md?

- **Skill** = called when needed, for a specific task.
- **CLAUDE.md** = always loaded, for universal standards.

3.3 Path-specific rules

Files in `.claude/rules/` have YAML frontmatter with a **`paths`** field. It contains glob patterns.

```
---
paths: ["terraform/**/*"]
---
```

The rule loads **only** when you edit a matching file. This keeps unrelated instructions out of the context and saves tokens.

The key comparison (Sample Q6)

When conventions apply to files **spread across many folders**, use glob rules. Do **not** use directory-level CLAUDE.md files.

The standard example: test files sitting next to their source code. `Button.test.tsx` is next to `Button.tsx`, in 60 different folders.

`paths: ["**/*.test.tsx"]` applies the conventions by **file type**, wherever the file is. A directory CLAUDE.md cannot do this, because it only covers one folder.

Why the other options in Q6 are wrong: - All conventions in the root CLAUDE.md, under headings -> the model must **guess** which section applies. Guessing is unreliable. - A skill per code type -> skills need to be **called**. The question asked for automatic. - A CLAUDE.md in each subfolder -> it only covers one folder, so it cannot handle files spread everywhere.

3.4 Plan mode or direct execution?

Plan mode means Claude explores and designs before making changes. **Direct execution** means Claude changes the code immediately.

Use plan mode	Use direct execution
Large changes	Small, clear changes
Several possible approaches	One obvious approach
Architectural decisions	A single-file bug fix with a clear stack trace
Changes across many files	Adding one validation check to one function
Prevents expensive rework	-

Examples from the guide

Plan mode: restructuring a monolith into microservices; a library migration affecting 45+ files; choosing between integration approaches that need different infrastructure.

Direct execution: a single-file bug fix with a clear stack trace; adding one date validation check.

Sample Q5 - why one answer is a trap

The trap answer says: "start with direct execution, and switch to plan mode if unexpected complexity appears."

This is wrong because **the complexity is already described in the requirements**. It is not something that might appear later.

Combining both

Use plan mode to investigate. Then use direct execution to implement the plan.

The Explore subagent

The **Explore subagent** handles noisy discovery work and returns only a summary. This keeps the main conversation clean during multi-phase tasks.

3.5 Iterative refinement

Technique	Use it when
Concrete input/output examples (2-3)	Written descriptions produce inconsistent results. The guide calls this the *most effective* way to show an expected transformation
Test-driven iteration	Write the tests first (normal behaviour, edge cases, performance). Then share the failures and improve step by step
The interview pattern	You are working in an unfamiliar area. Let Claude ask you questions first, so it raises things you did not think about - for example cache invalidation or failure modes
Specific test cases with input and expected output	Fixing an edge case, such as null values in a migration script

One message or several?

- Problems that **affect each other** -> put them all in one detailed message.
- Problems that are **independent** -> fix them one at a time.

3.6 Claude Code in CI/CD

The command-line flags

Learn these exactly.

Flag	What it does
`-p` / `--print`	Non-interactive mode. **This fixes a pipeline that hangs** (Sample Q10)
`--output-format json`	Output that a script can read
`--json-schema`	Forces the output to follow a schema

These do not exist. The exam offers them as wrong answers: - `--batch` - `CLAUDE_HEADLESS=true` - Redirecting stdin from `/dev/null` as the "correct approach"

Use `--output-format json` together with `--json-schema` to produce structured findings you can post automatically as inline pull request comments.

CLAUDE.md gives CI the project context

When Claude Code runs in CI, CLAUDE.md is how it learns your testing standards, fixture conventions, and review criteria. Writing these down reduces low-value test suggestions.

Session context isolation

The same session that **wrote** code is worse at reviewing that code. It remembers its own reasoning and does not question it.

Use a **separate, independent instance** to review. (This also appears in section 4.6.)

Re-running a review after new commits

Include the earlier findings in the context. Tell Claude to report only **new or still-unfixed** issues. This stops duplicate comments.

Generating tests

Give Claude the existing test files. Then it will not suggest tests for scenarios that are already covered.

Memory drill for Domain 3

Cover the right column. Say the answer out loud. Do this every day.

Question	Answer
Shared team slash command	`.claude/commands/`
Personal slash command	`~/.claude/commands/`
Not shared through version control	`~/.claude/CLAUDE.md`
Keep CLAUDE.md modular	`@import`
Rule files by topic	`.claude/rules/`
Load a rule only for matching files	YAML frontmatter `paths:` with globs
Keep a noisy skill out of the main conversation	`context: fork`
Limit which tools a skill can use	`allowed-tools`
Ask for a missing parameter	`argument-hint`
Which memory files are loaded?	`/memory`
Reduce context in a long session	`/compact`
Non-interactive mode in CI	`-p` / `--print`

Structured output in CI	`--output-format json` + `--json-schema`
Noisy discovery without filling the context	Explore subagent
Test conventions across many folders	`.claude/rules/` glob, not a directory CLAUDE.md

Domain 4 - Prompt Engineering & Structured Output (20%)

About 12 of the 60 questions come from here. There are 6 task statements.

This is your weakest domain. You scored 1 out of 5 in the day-1 test. The three questions you missed were all facts, not reasoning. Facts are quick to fix. Use `practice/daily-facts-domain-4.md` every day.

Two ideas run through this domain:

1. **Clear, specific criteria work. Vague instructions do not.**
2. **Tool use with a JSON schema is the reliable way to get structured output.**

4.1 Write specific criteria, not vague instructions

[no] Vague	[done] Specific
"Check that comments are accurate"	"Flag a comment only when the behaviour it describes **contradicts** what the code does"
"Be conservative"	Name the categories to report and the categories to skip
"Only report high-confidence findings"	Report bugs and security issues. Skip minor style and local patterns

Filtering by confidence does not work

Instructions like "be conservative" or "only report what you are sure about" do **not** improve precision. Specific categories do.

On the exam, any answer that filters by the model's own confidence is probably wrong.

False alarms damage trust in everything

If one category produces many false alarms, developers stop trusting the **accurate** categories too.

The fix from the guide: temporarily turn off the category with many false alarms. Keep the good categories running. Improve the bad category's prompt, then turn it back on.

Consistent severity levels

Define each severity level with **concrete code examples**. Then classification stays consistent.

4.2 Few-shot prompting

Few-shot means putting a small number of examples in the prompt.

The guide says few-shot examples are the most effective technique when you need consistent, well-formatted output and detailed instructions alone are not working.

The rules

- Use **2 to 4 examples**.
- Aim them at the **unclear cases**, not the obvious ones.

What few-shot examples achieve

- They show how to handle **unclear situations** (which tool to pick for an ambiguous request; gaps in branch-level test coverage).
- They let the model **apply the same judgement to new situations** it has not seen. It does not only copy the examples.
- They **reduce invented information** in extraction tasks (informal measurements, documents with different structures).
- They show the **reasoning** for choosing one action instead of another reasonable one.

- They fix the **output format** (location, issue, severity, suggested fix).
- They separate **acceptable code patterns from real problems**, which reduces false alarms.
- They handle **different document structures**: inline citations vs a bibliography; a methodology section vs details spread through the text.
- They fix **empty or null values in required fields**, by showing correct extraction from documents with different layouts.

When few-shot is the wrong answer

- When the real problem is a **tool description**. Sample Q2 says few-shot "adds token overhead without fixing the underlying issue."
- When the rule **must always work**. Sample Q1: few-shot is still probabilistic, so it fails sometimes.

4.3 Structured output with tool use and JSON schemas

Using **tool use with a JSON schema** is the most reliable way to guarantee output that matches your schema. It removes JSON syntax errors.

The important detail

Strict schemas remove **syntax** errors. They do **not** remove **semantic** errors.

- **Syntax error:** the JSON is broken. Schemas fix this.
- **Semantic error:** the JSON is valid, but the content is wrong. Line items do not add up to the total. A value is in the wrong field. Schemas do **not** fix this.

Any answer claiming schemas guarantee correct content is wrong.

`tool_choice` again, in this context

Value	What happens	How the exam uses it
`auto`	The model **may return text** instead of calling a tool	The default. Never the fix when the model returns text instead of extracting
`any`	Must call a tool, chooses which	**You have several schemas and do not know the document type yet**
`{"type":"tool","name":"..."}`	Must call that specific tool	**Force `extract_metadata` to run before enrichment**, then continue in later turns
`none`	Cannot use tools	Real, but the guide does not test it

Schema design rules

Make fields optional or nullable when the document might not contain them.

This is the most tested schema fact. A **required** field forces the model to produce something. If the value is not in the document, the model invents one.

Other rules: - Use an enum with an **`"other"` value plus a detail string** for categories that may grow. - Add an **`"unclear"`** enum value for ambiguous cases. - Put **format normalisation rules in the prompt**, next to the strict schema. This handles source documents with inconsistent formatting.

4.4 Validation, retries, and feedback

How to retry correctly

When validation fails, send a follow-up request containing three things: 1. The original document 2. The failed extraction 3. The **specific** validation errors

Then the model can correct itself.

When retries work, and when they do not

Retry works	Retry does not work
Format is wrong (dates, currency)	The information is **not in the document at all**
The output structure is wrong	The information is only in another document you did not provide

This is simple but easy to forget. Format problems are fixable. Missing information is not.

Two kinds of validation error

- **Semantic errors** - values do not add up, a value is in the wrong field. Tool use does not prevent these.
- **Syntax errors** - broken JSON. Tool use does prevent these.

Two schema patterns for self-checking

- Extract `calculated_total` **next to** `stated_total`. If they differ, you have found a problem.
- Add a `conflict_detected` boolean for source data that contradicts itself.

Analysing false alarms

Add a `detected_pattern` field to each finding. It records which code pattern triggered the finding. Later you can look at which patterns developers usually dismiss.

4.5 The Message Batches API

Learn this table.

Property	Value
Cost	**50% cheaper**
Processing time	**Up to 24 hours**
Guaranteed speed	**None**
Matching responses to requests	`custom_id`
Tool calling across several turns in one request	**Not supported**

Good for: work that can wait - overnight reports, weekly audits, nightly test generation.

Bad for: work that blocks someone - a pre-merge check where a developer is waiting.

Sample Q11

Use batch for the overnight technical-debt report. Keep real-time for the blocking pre-merge check.

The wrong answers: - Batch both, because batches are "often faster" -> not acceptable when someone is waiting - Keep both real-time because of result ordering -> wrong; `custom_id` matches results correctly - Batch both with a timeout fallback -> unnecessary complexity

Three more batch skills

1. **Calculate how often to submit**, based on your SLA. Example: with a 24-hour processing window, submit every 4 hours to guarantee a 30-hour SLA.

2. **Resubmit only the failed documents.** Find them by `custom_id`. Change them first if needed - for example, split documents that were too long.
3. **Test your prompt on a small sample first**, before sending a large batch. This raises the first-attempt success rate and reduces the cost of resubmitting.

(Extra detail beyond the guide: results come back in any order, so match them by `custom_id`, never by position. A batch can hold up to 100,000 requests or 256 MB. Results are kept 29 days.)

4.6 Reviewing with several passes or several instances

Why a model reviewing its own work is weak

It still remembers why it wrote the code that way. So it does not question its own decisions.

What works better

A **separate, independent instance** with no memory of writing the code. This works better than telling the model to check itself, and better than extended thinking.

Multi-pass review for large changes

- **One pass per file** -> finds local issues, with consistent depth
- **One separate pass across files** -> finds data-flow issues between files

This prevents **attention dilution** (quality dropping when there is too much to review at once) and contradictory findings - for example, flagging a pattern as a bug in one file while approving the same pattern in another file of the same pull request.

Sample Q12 - the wrong answers

- A bigger model or bigger context window -> "larger context windows don't solve attention quality."
- Ask developers to split their pull requests -> moves the work to people, does not fix the system.
- Run three passes and report only issues found in at least two -> **hides real bugs** that are only caught sometimes.

One place where confidence is allowed

In a verification pass, the model can report a confidence level **next to each finding**. This helps you decide which findings a human should look at first.

Compare with section 5.2, where self-reported confidence is **rejected** as a reason to escalate. The difference: here it directs a reviewer's attention. There it decides whether the agent acts alone.

Quick review list for Domain 4

1. Specific categories beat "be conservative" and confidence filters.
2. A category with many false alarms damages trust in the good ones. Turn it off while you fix it.
3. Few-shot: 2 to 4 examples, aimed at unclear cases. They allow the model to generalise.
4. Tool use + JSON schema removes **syntax** errors, not **semantic** ones.
5. `auto` may return text - `any` guarantees a tool call - forced picks the tool.
6. Optional or nullable fields stop invented values. Use `"other"` + detail, and `"unclear"`.
7. Retry with: the document + the failed extraction + the specific errors. Retry cannot find missing information.
8. `calculated_total` vs `stated_total`; `conflict_detected`; `detected_pattern`.
9. Batches: 50% cheaper, up to 24 hours, no speed guarantee, `custom_id`, no multi-turn tool calling.
10. Independent instance beats self-review. One pass per file + one pass across files.

Domain 5 - Context Management & Reliability (15%)

About 9 of the 60 questions come from here. There are 6 task statements.

This is the smallest domain by weight. But it appears in **four of the six exam scenarios**, so you meet it often.

5.1 Keeping important information during long conversations

Three problems to know by name

1. Progressive summarization risk When you keep summarising a conversation, you lose the details: numbers, percentages, dates, and what the customer said they expected. Those details are exactly what you need later.

2. The "lost in the middle" effect Models handle the **beginning** and the **end** of a long input well. They may miss things in the **middle**.

3. Tool results fill up the context Tool output uses many tokens, and most of it is not useful. The guide's example: an order lookup returns **more than 40 fields** when only 5 matter.

The fixes

Problem	Fix
Summarising loses details	Pull the facts (amounts, dates, order numbers, statuses) into a **"case facts" block**. Include it in every prompt, **outside** the summarised history
Sessions with several issues	Pull structured issue data (order IDs, amounts, statuses) into a **separate context layer**
Tool output is too long	**Trim it to the useful fields before it enters the context.** Do this early, not later
Lost in the middle	Put a **summary of key findings at the beginning**. Organise the details with clear section headings
Later agents lose meaning	Require subagents to include **metadata** in their output: dates, source locations, methodology
A later agent has little context space	Change the **earlier** agents to return **structured data** (key facts, citations, relevance scores) instead of long text and reasoning

Also: the API has no memory. Send the full conversation history with every request.

5.2 When to escalate to a human

Three valid reasons to escalate

1. The customer **asks for a human**.
2. There is a **policy gap or exception** - the policy does not cover this request.
3. The agent **cannot make progress**.

Note the second one carefully. It is *policy gaps*, not simply "difficult cases."

Two signals that are always wrong

- **Sentiment analysis.** How angry the customer sounds does not tell you how difficult the case is.
- **The model's own confidence score.** It is not well calibrated. The agent is already overconfident on the hard cases it is getting wrong.

These two appear as wrong answers throughout the exam.

Specific situations to remember

Situation	Correct behaviour
The customer clearly asks for a human	Escalate **immediately**. Do not investigate first
The customer is frustrated, but the agent can solve the problem	Acknowledge the frustration, offer to solve it. Escalate only if the customer **asks again**
The policy does not mention this case (example: matching a competitor's price, when the policy only covers your own site)	Escalate
A tool returns several matching customers	**Ask for another identifier.** Never guess

The first two rows look similar and are easy to confuse. The difference: did the customer **explicitly ask** for a human?

Fixing bad escalation decisions (Sample Q3)

Add explicit escalation criteria to the system prompt, **with few-shot examples** showing when to escalate and when to solve.

Not a classifier. Not sentiment. Not self-reported confidence.

5.3 Reporting errors between agents

When a subagent fails, it should return **structured error context** so the coordinator can decide what to do.

Four parts: 1. **What kind of failure it was** 2. **What it tried** (the query) 3. **Any partial results** 4. **Other approaches that might work**

With this, the coordinator can retry with a different query, try another route, or continue with partial results.

Four wrong approaches

Wrong approach	Why it is wrong
A generic message like "search unavailable"	Hides useful information from the coordinator
Hiding the error - return empty results and call it a success	Recovery becomes impossible, and the final output looks complete when it is not
Stopping the whole workflow because one agent failed	Unnecessary. Other approaches might still work
Treating an **access failure** and a **valid empty result** as the same	A timeout may need a retry. Zero matches is a successful search

Local recovery

A subagent fixes transient failures itself. It only reports what it cannot fix - and when it does, it includes what it tried and any partial results.

Coverage annotations

The synthesis output should say which findings are **well supported** and which topics have **gaps**, because sources were unavailable.

5.4 Managing context while exploring a large codebase

The symptom to recognise

In long sessions, the model starts giving **inconsistent answers**. It talks about **"typical patterns"** instead of the specific classes it found earlier.

If an exam question describes this, it is asking about context degradation.

Five techniques

Technique	What it does
Scratchpad files	Save key findings to a file. Read the file later. This is the direct fix for context degradation
Subagent delegation	Send noisy exploration to a subagent ("find all test files", "trace the refund flow"). The main agent keeps the high-level picture
Phase summaries	Summarise one phase before starting the next. Put the summary into the next agents' starting context
Manifests	For crash recovery. Each agent saves its state to a known location. The coordinator loads the manifest when work resumes, and puts it into the agents' prompts
`/compact`	Reduces context usage during a long session filled with discovery output

---

5.5 Human review and confidence calibration

The main risk

An average accuracy number can hide a failing part. 97% overall accuracy can hide one document type or one field performing very badly.

So: check accuracy **by document type and by field** before you reduce human review.

Four techniques

Stratified random sampling of high-confidence extractions. You sample the high-confidence group on purpose. That is the group you are about to stop reviewing, so you need to know its real error rate. It also finds new kinds of error.

Accuracy analysis by document type and field. Confirm every segment performs well, not just the average.

Field-level confidence scores, calibrated with labelled validation sets. Calibration against labelled data is what makes confidence usable here.

Route to human review when model confidence is low, or when the source document is ambiguous or contradicts itself. This uses limited reviewer time well.

How this fits with section 5.2

- Self-reported confidence is **rejected** as a reason to escalate or act alone.
- It is **accepted** for deciding which items a reviewer should check first - but only when it is **calibrated with labelled data**.

Uncalibrated confidence is never the right answer.

5.6 Keeping track of sources

How source information gets lost

It is lost during **summarisation**, when findings are compressed without keeping the link between each claim and its source.

What to require

- Subagents must output **structured claim-source mappings**: source URL, document name, and the **relevant excerpt**. Later agents must keep and merge these.
- Subagents must include **publication or collection dates**. Without dates, two numbers from different years look like a contradiction when they are not.

When two credible sources disagree

- **Record both values with their sources.** Never pick one at random.
- Document analysis should finish with **both conflicting values included and clearly marked**. The **coordinator** then decides how to handle them, before synthesis runs.
- Structure the report with separate sections for **well-established findings** and **contested findings**. Keep the original wording and methodology of each source.

Formatting the output

Use the right format for each type of content: - Financial data -> **tables** - News -> **prose** - Technical findings -> **structured lists**

Do not convert everything into one format.

Quick review list for Domain 5

1. Keep a "case facts" block in every prompt, outside the summarised history.
2. Lost in the middle -> key findings first, clear section headings.
3. Trim tool output **before** it enters the context (40+ fields -> 5).
4. Escalate when: the customer asks, the policy has a gap, or progress stops. Never sentiment. Never self-reported confidence.
5. Several matching customers -> ask for another identifier. Do not guess.
6. Error context = failure type + what was tried + partial results + alternatives.
7. Never: generic error messages, hidden errors, stopping the whole workflow, or treating an access failure as an empty result.
8. "Typical patterns" instead of specific classes = context degradation -> use scratchpad files.
9. Crash recovery = manifests, loaded by the coordinator when work resumes.
10. 97% average can hide a broken segment -> stratified sampling + analysis by type and field.
11. Confidence is usable only when **calibrated with labelled data**, and only for routing review.
12. Conflicts -> record both with sources. Dates are required. Tables, prose, or lists by content type.

The Six Exam Scenarios

The exam gives you **4 of these 6**, chosen at random. Each one is a short story, followed by about 15 questions.

> **Read this once. Do not memorise it.**

> A candidate who passed CCAR-F describes the scenarios as *"more dressing than anything substantive... no different to an exam that just asked me a series of unrelated questions about specific topics."*

> That matches the twelve sample questions in the official guide: every one of them is **self-contained**. You never need to remember a detail from the scenario text to answer a question.

> So this file has one job: **prediction, not memorisation.** When you recognise the scenario, you know which domains the questions will come from and which wrong answers to expect. That is worth 20 minutes of reading. It is not worth an hour of study.

> If you are short of time, skip this file and spend the time on `03-cheat-sheet.md` instead.

Scenario 1 - Customer Support Resolution Agent

Built with the Claude Agent SDK. It handles difficult requests: returns, billing disputes, account problems. It uses four MCP tools: `get_customer`, `lookup_order`, `process_refund`, `escalate_to_human`. The goal is **80%+ first-contact resolution**, while knowing when to pass a case to a human.

Domains: 1 (Agentic Architecture), 2 (Tool Design & MCP), 5 (Context & Reliability)

Expect questions about: - Blocking `process_refund` until identity is verified -> programmatic enforcement - The agent confusing `get_customer` and `lookup_order` -> tool descriptions - Escalating the wrong cases -> explicit criteria + few-shot. Never sentiment or confidence - Refunds above a limit -> tool call interception hook ($500) - Several matching customers -> ask for another identifier - A "case facts" block for amounts, dates, order numbers - A structured handoff summary for the human agent

Quick answers: money -> hooks. Wrong tool -> descriptions. Escalation -> criteria + few-shot. Long order results -> trim the fields.

Scenario 2 - Code Generation with Claude Code

A team uses Claude Code for writing code, refactoring, debugging, and documentation. They need custom slash commands, CLAUDE.md configuration, and to know when to use plan mode.

Domains: 3 (Claude Code Configuration), 5 (Context & Reliability)

Expect questions about: - Where a shared `/review` command lives -> `.claude/commands/` - Why a teammate does not get the instructions -> user-level CLAUDE.md - Conventions for test files spread across folders -> `.claude/rules/` with globs - Plan mode for microservice restructuring or a library migration - Direct execution for a single-file fix - `context: fork`, `allowed-tools`, `argument-hint` in SKILL.md - `/memory`, `/compact`, the Explore subagent, scratchpad files

Quick answers: "every developer when they clone" -> project scope. "Spread throughout" -> glob rules. "Architectural" or "dozens of files" -> plan mode.

Scenario 3 - Multi-Agent Research System

A coordinator gives work to specialised subagents: web search, document analysis, synthesis, and report generation. The system produces detailed reports **with citations**.

Domains: 1 (Agentic Architecture), 2 (Tool Design & MCP), 5 (Context & Reliability)

Expect questions about: - The report misses part of the topic -> the coordinator's split was too narrow - A subagent times out -> structured error context, not a generic message - Synthesis needs to check facts often -> a small `verify_fact` tool -

Running subagents in parallel -> several Task calls in one response; `allowedTools` includes `"Task"` - Keeping claim-source links through synthesis - Two sources disagree -> record both with their sources - Publication dates are required, so old and new numbers are not mistaken for a contradiction - Marking coverage gaps in the final report

Quick answers: missing coverage -> the coordinator. A failure -> structured context + partial results. A conflict -> record both, never choose one.

Scenario 4 - Developer Productivity with Claude

An agent helps engineers explore unfamiliar codebases, understand old systems, generate boilerplate, and automate repetitive work. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and MCP servers.

Domains: 2 (Tool Design & MCP), 3 (Claude Code Configuration), 1 (Agentic Architecture)

Expect questions about: - Grep or Glob? - Edit cannot find unique text -> use Read + Write - Exploring step by step: Grep to find entry points, then Read to follow imports - MCP scoping: `.mcp.json` or `~/.claude.json`, and `${VAR}` expansion - The agent prefers built-in Grep over a better MCP tool -> improve the MCP tool description - MCP **resources** as content catalogues, to avoid exploratory calls - A community MCP server or a custom one? (Jira -> use the community server) - Long sessions losing detail -> scratchpad files, subagents, `/compact` - Crash recovery with manifests

Quick answers: inside files -> Grep. File paths -> Glob. Edit fails -> Read + Write. Standard integration -> community server.

Scenario 5 - Claude Code for Continuous Integration

Claude Code runs inside a CI/CD pipeline: automatic code review, test generation, and pull request feedback. The feedback must be useful, with **few false alarms**.

Domains: 3 (Claude Code Configuration), 4 (Prompt Engineering & Structured Output)

Expect questions about: - The pipeline hangs -> `-p` / `--print` - Output a script can read -> `--output-format json` + `--json-schema` - Too many false alarms -> specific criteria, not "be conservative" - Inconsistent depth on a 14-file pull request -> one pass per file + one pass across files - Batch or real-time? (blocking check = real-time; overnight report = batch) - Duplicate comments after a new commit -> include earlier findings, report only new ones - Duplicate tests -> give it the existing test files - CLAUDE.md holding testing standards and fixture conventions - Reviewing code the same session wrote -> use an independent instance - A `detected_pattern` field to analyse dismissed findings

Quick answers: hangs -> `-p`. Inconsistent depth -> split the passes. False alarms -> specific criteria. Someone is waiting -> real-time.

Scenario 6 - Structured Data Extraction

The system extracts information from unstructured documents, checks it with JSON schemas, keeps accuracy high, handles edge cases, and sends results to other systems.

Domains: 4 (Prompt Engineering & Structured Output), 5 (Context & Reliability)

Expect questions about: - The model invents values for required fields -> make the fields optional and nullable - Unknown document type with several schemas -> `tool_choice: "any"` - Forcing `extract_metadata` to run first -> forced tool selection - Enum with `"other"` + detail; `"unclear"` for ambiguous cases - Retry with specific errors; and when retry cannot help (information is not in the document) - Semantic vs syntax errors (`calculated_total` vs `stated_total`, `conflict_detected`) - A batch of 100 documents, `custom_id`, resubmitting only the failures, splitting long documents - SLA arithmetic (submit every 4 hours for a 30-hour SLA) - 97% average accuracy hiding one bad document type - Stratified sampling of high-confidence extractions - Field-level confidence, calibrated with labelled data - Few-shot examples for documents with different structures

Quick answers: invented values -> nullable. Unknown type -> `any`. Missing information -> retry will not help. 97% -> analyse by segment.

One pattern across all six

Domain 5 appears in four of the six scenarios, even though it is only 15% of the exam. Reliability ideas connect everything.

So if you are stuck between two answers, ask which one handles reliability better: structured errors, preserved facts, or calibrated human review. That often decides it.

What this means on exam day

Because the questions are self-contained, **do not spend four minutes reading each scenario story.** Skim it for the tool names and the goal, then go straight to the questions. If a question needs a detail from the story, that detail is repeated in the question itself.

A candidate who passed finished the whole exam in **90 of the 120 minutes**. Time is adequate but not generous. Reading four stories carefully would spend a quarter of it for no gain.

Domain 4 - Daily Fact Drill (5 minutes)

Why this file exists: your day-1 diagnostic scored 1/5 in Domain 4, on 20% of the exam. The three misses (Q13, Q14, Q15) were flat recall failures, not reasoning failures - so the treatment is spaced repetition, not more study time.

How to use it: once a day, every day, from day 2 to day 19. Cover the right column, recite aloud, uncover. Under 5 minutes. Mark anything you miss twice in a row with a [n] and drill only those the next day.

Do **not** skip days. Eighteen 5-minute reps beat one 90-minute session; that's the entire point.

Block A - Schema design (Q13's block)

Prompt	Answer
Model fabricates plausible values for missing data. Cause?	The field is marked **required** in the schema
Fix?	Make it **optional / nullable**
Why does required cause fabrication?	The model must produce *something* to satisfy the schema
Extensible category that may not fit the enum?	Enum value `"other"` **+ a detail string field**
Ambiguous case, can't determine the value?	Enum value `"unclear"`
Source formatting is inconsistent (dates, currency)?	**Format normalization rules in the prompt**, alongside the strict output schema
Strict schemas eliminate which errors?	**Syntax** errors only
Which errors do they NOT prevent?	**Semantic** - line items that don't sum, values in the wrong fields
Detect a total mismatch?	Extract **`calculated_total`** **alongside** **`stated_total`**
Flag inconsistent source data?	A **`conflict_detected`** boolean
Analyze which code constructs trigger dismissed findings?	A **`detected_pattern`** field on each finding

---

Block B - `tool_choice` (Q14's block)

Value	Behavior	Use it when
`{"type": "auto"}`	Model decides; **may return text instead of calling a tool**	Default. Never the fix for "it returned text instead of extracting"
`{"type": "any"}`	**Must** call a tool, chooses which	Multiple schemas, document type unknown; guarantee structured output
`{"type": "tool", "name": "..."}`	Must call **that** tool	Force `extract_metadata` before enrichment, then continue in follow-up turns
`{"type": "none"}`	Cannot use tools	Exists in the API; not tested by the guide

One-line version: **auto may talk - any must act - forced picks the actor.**

Block C - Retry limits (Q15's block)

Prompt	Answer
Retry **works** for which failures?	**Format mismatches** and **structural output errors**

Retry is **useless** for which?	The information is **absent from the source document** (e.g. exists only in a document not supplied)
What goes in the retry request?	Original document **+** the failed extraction **+** the **specific** validation errors
Which error class does tool use already eliminate?	Schema **syntax** errors
Which class survives and needs validation?	**Semantic** errors

---

Block D - Message Batches API

Prompt	Answer
Cost	**50%** savings
Processing window	up to **24 hours**
Latency SLA	**None guaranteed**
Request/response correlation	`custom_id`
Multi-turn tool calling in one request?	**Not supported**
Appropriate for?	Non-blocking, latency-tolerant: overnight reports, weekly audits, nightly test generation
Inappropriate for?	Blocking workflows - pre-merge checks, pre-commit hooks
A batch fails on some documents. Resubmit what?	**Only the failed ones**, identified by `custom_id`, with modifications (chunk oversized docs)
Guarantee a 30-hour SLA with a 24-hour window?	Submit every **4 hours**
Before batching 10,000 documents?	**Refine the prompt on a sample set** to maximize first-pass success

---

Block E - Criteria and few-shot

Prompt	Answer
"Be conservative" / "only high-confidence findings" - effect on precision?	**None.** Vague instructions don't improve precision
What does work?	**Specific categorical criteria**: which issue classes to report (bugs, security) vs skip (minor style)
Example of vague vs explicit	"Check comments are accurate" -> "Flag comments only when claimed behavior **contradicts** actual code behavior"
One category has 40% false positives, another 5%. Effect?	The bad category **undermines trust in the accurate one**
Remedy?	**Temporarily disable** the high-FP category while improving its prompt
Consistent severity classification?	Explicit severity criteria **with concrete code examples per level**
How many few-shot examples?	**2-4**, targeted
Target them at which cases?	The **ambiguous** ones, not the obvious ones

What should each example show?	The **reasoning for choosing one action over plausible alternatives**
Key benefit beyond format?	**Generalization to novel patterns**, not just matching pre-specified cases
When does few-shot lose?	When the root cause is a **tool description** problem, or when **deterministic compliance** is required

Block F - Multi-pass review

Prompt	Answer
Why is self-review weak?	The model **retains reasoning context from generation** and won't question its own decisions
Better than self-review instructions or extended thinking?	An **independent review instance** with no prior reasoning context
14-file PR, inconsistent depth and contradictory findings. Cause?	**Attention dilution**
Fix?	**Per-file local passes + a separate cross-file integration pass**
Does a bigger context window fix it?	**No** - larger windows don't solve attention quality
Does 3-runs-flag-if-2-agree fix it?	**No** - suppresses real bugs caught intermittently
Is self-reported confidence ever acceptable?	Yes - **alongside each finding**, to route reviewer attention. Never as an autonomy/escalation trigger

Weekly self-test

Every 7th day, write these out from a blank page instead of reading:

1. The four `tool_choice` values and one exam use for each.
2. Why required schema fields cause fabrication, and the fix.
3. The two retryable error classes and the one non-retryable one.
4. Five facts about the Message Batches API.
5. Why "be conservative" fails and what replaces it.
6. Two things that do **not** fix attention dilution.

If you can do all six cold by day 14, Domain 4 is no longer your weak domain.

Hands-On Exercises

These are the four exercises from the exam guide, turned into practical work. Each step has a **checkpoint** [done] showing what the exam tests.

The goal is not a finished product. The goal is to turn "I recognise this" into "I remember this."

How to work through these

You have built things like this before with AI help. For exam preparation, do it the other way round: **write the configuration and schema by hand first, then let Claude check it.**

The exam is proctored. You get no notes and no assistant. Typing `context: fork` yourself three times helps more than reading it thirty times.

Exercise 1 - Agent with several tools and escalation (Days 2-6)

Domains: 1, 2, 5

Build a small support agent. Suggested folder: `prep/exercises/ex1-agent/`

Steps

1. Define 3-4 MCP tools with detailed descriptions. Include **two tools that do similar things** (for example `get_customer` and `lookup_order`), and write descriptions that make the difference clear.

- [done] Each description says: input formats, example queries, edge cases, and **when to use it instead of the similar tool**.
- Then try the opposite. Shorten both descriptions to one line and send unclear requests. Watch the agent choose wrongly. That is Sample Q2 happening in front of you.

2. Build the agentic loop, controlled by `stop_reason`.

- [done] Your loop checks `"tool_use"` and `"end_turn"`. It does **not** read the text, and does not use a loop counter as the main way to stop.
- [done] Tool results go into the conversation history. All results from one Claude message go back in **one** user message, each with the matching `tool_use_id`.

3. Add structured error responses: `errorCategory` (transient, validation, permission, business), `isRetryable`, and a readable message.

- [done] Cause each error type on purpose. Check that the agent retries transient errors, explains business errors, and does not retry the others.
- [done] Your tools tell the difference between an **access failure** and a **valid empty result**.

4. Add a tool call interception hook that blocks an action above a limit (for example refunds over $500) and sends it to escalation.

- [done] Test it while the conversation is actively arguing for the refund. The block must still hold. That is the point of programmatic enforcement.

5. Test a message with two problems in it.

- [done] The agent splits it into separate items, handles each one, and gives one combined answer.

Memory check after finishing

Without looking, say: the four error categories, the two `stop_reason` values your loop uses, and the two hook types (change results vs block calls).

Exercise 2 - Configure Claude Code for a team (Days 7-9)

Domains: 3, 2

Do this in a **real repository**. Use one of your existing project folders.

This exercise gives the most points per hour. Domain 3 is 20% of the exam and is almost entirely memorisation.

Steps

1. A project-level CLAUDE.md with coding and testing standards. - [done] Say out loud why this is not in `~/.claude/CLAUDE.md`.

2. Two files in `.claude/rules/` with YAML frontmatter:

```
---
paths: ["src/api/**/*"]
---
```

Make the second one match by **file type across folders**: `paths: ["**/*.test.*"]`. - [done] Open a matching file and a non-matching file. Check the rule loads only for the match. - [done] Say why the second one could not be a subfolder CLAUDE.md.

3. A project skill in `.claude/skills/` using `context: fork` **and** `allowed-tools`. - [done] Check the skill's output does not appear in the main conversation. - Add `argument-hint` and call the skill with no arguments, to see the prompt.

4. A project slash command in `.claude/commands/`. - [done] Say why not `~/.claude/commands/`.

5. MCP configuration: a project server in `.mcp.json` using `${VAR}` for its token, plus a personal server in `~/.claude.json`. - [done] Check that **both** work at the same time. - [done] Check that no secret is committed.

6. Plan mode vs direct execution on three tasks: a single-file bug fix, a multi-file migration, and a feature with several possible designs. - [done] After each one, write one sentence saying why that mode was correct.

7. Run `/memory` and read the output. Run `/compact` in a long session.

Memory check

Write from memory: the three CLAUDE.md levels and their paths; the four folders under `.claude/`; the three SKILL.md frontmatter keys; the three CI flags.

Exercise 3 - Structured extraction pipeline (Days 11-13)

Domains: 4, 5

Steps

1. Define an extraction tool with a JSON schema containing: required fields, **optional/nullable** fields, and an **enum with `"other"` plus a detail string**.

- [done] Process documents that are missing the optional fields. Check the model returns `null` instead of inventing a value.
- [done] Now change one field to required. Watch the model invent a value. Seeing both is the lesson.

2. Build a validation-retry loop. On failure, send back the document, the failed extraction, and the **specific** validation error.

- [done] Sort your failures into two groups: retryable (format or structure) and not retryable (information missing from the document). Count them. This distinction is tested directly.

3. Add few-shot examples for documents with different structures (data inline, in a table, in prose).

- [done] Use 2 to 4 examples, aimed at the **unclear** cases, not the obvious ones.

4. Add semantic validation. Extract `calculated_total` next to `stated_total`. Add a `conflict_detected` boolean.

- [done] Create a case where the schema passes but the data is wrong. Now you have felt why "schemas do not prevent semantic errors."

5. Batch processing with the Message Batches API. Submit a batch, match results by `custom_id`, and resubmit only the failures (split any document that was too long).

- [done] Do the SLA arithmetic: with a 24-hour window, how often must you submit to guarantee a 30-hour SLA?
- [done] Confirm results are matched by `custom_id`, not by position.

6. Human review routing. Add field-level confidence scores. Send low-confidence and contradictory documents to review. Analyse accuracy **by document type and by field**.

- [done] Create a case where the average accuracy looks good but one document type is failing badly.

Exercise 4 - Multi-agent research pipeline (Days 14-15)

Domains: 1, 2, 5

Steps

1. A coordinator plus two subagents (web search, document analysis). - [done] The coordinator's `allowedTools` includes `"Task"`. - [done] Each subagent gets its input **in its prompt**. Test this: ask a subagent about something the coordinator discussed earlier. It should not know.

2. Parallel execution: send several Task calls in **one** response. Compare the time with running them one after another. - [done] You can explain the difference between this and `fork_session`.

3. Structured subagent output separating content from metadata: claim, evidence excerpt, source URL or document name, publication date. - [done] The source information survives synthesis.

4. Error reporting: simulate a subagent timeout. - [done] The coordinator receives: failure type, the query it tried, partial results, and possible alternatives. It continues with partial results and marks the coverage gap.

5. Conflicting sources: give it two credible sources with different numbers. - [done] The synthesis keeps both, with their sources, and separates well-established findings from contested ones.

6. Extra - recreate the Sample Q7 failure on purpose. Give the coordinator a broad topic and a prompt that pushes it toward one narrow area. Watch every subagent succeed while the report still misses most of the topic.

This takes five minutes and teaches more than any reading.

If you only have time for two

Do **Exercise 2** and **Exercise 4**.

- Exercise 2: Domain 3 is 20% of the exam and pure memorisation.
- Exercise 4: Domain 1 is 27% and the hardest to guess your way through.

Exercises 1 and 3 repeat much of the same material on error handling and schema design.

The Recommended Courses - Which to Take, Which to Skip

The Partner Academy recommends seven courses.

- **Two are not tested on this exam at all.**
- **Two are too basic for this exam.**
- **One is only partly useful.**
- **Two are worth your time.**

Choosing correctly saves you several days.

What decides this: Section 6 (Detailed Objectives) and Section 17 (In-Scope and Out-of-Scope Topics) of the exam guide. Not the course catalogue.

[done] Take - Claude Code in Action (Level 200)

> *Integrate Claude Code into development workflows - context management, hooks, custom commands, and the Agent SDK.*

This is the most useful course on the list. Its four topics cover three exam domains:

Course topic	Domain	Weight
Custom commands, workflows	3 - Claude Code Configuration	20%
Hooks, Agent SDK	1 - Agentic Architecture	27%
Context management	5 - Context & Reliability	15%

That is about **62% of the exam**. Take this one first.

Pay attention to these - all are tested: - `PostToolUse` hooks (change results) vs interception hooks (block calls) - The Task tool, and `allowedTools` needing `"Task"` - `.claude/commands/` vs `~/.claude/commands/` - SKILL.md frontmatter: `context: fork`, `allowed-tools`, `argument-hint` - `-p` / `--print`, `--output-format json`, `--json-schema` - `/memory`, `/compact`, `--resume`, `fork_session`, the Explore subagent

This course replaces: most of Exercise 2, and the hooks part of Exercise 1.

[done] Take - Introduction to Model Context Protocol (Level 200)

> *Build MCP servers and clients from scratch using Python. Master tools, resources, and prompts to connect Claude with external services.*

This covers Domain 2 (18%).

Note the word **resources** in the description. The exam tests MCP resources specifically: they show the agent **content catalogues** (issue lists, documentation structures, database schemas) so it does not need exploratory tool calls. Most people learn MCP tools and ignore resources. Do not skip that part.

Pay attention to: - The `isError` flag and structured error responses - How description quality affects tool selection - Server configuration

The course probably will not cover these tested items. Get them from the notes: - `.mcp.json` (project) vs `~/.claude.json` (user) - `${VAR}` environment variable expansion - The four error categories: transient, validation, business, permission - The "18 tools vs 4-5" guidance

This course replaces: the tool-design half of Exercise 1.

! Take only parts - Building with the Claude API (Level 100-200)

> *Authentication, prompt engineering, and evaluations through tool use, RAG, agents, and production patterns.*

Three of its six topics are **not tested**:

Topic	Status
Authentication	[no] Out of scope: "Claude API authentication, billing, or account management"
RAG	[no] Not in this exam. It belongs to the Professional exam
Evaluations	[no] Not in this exam. It belongs to the Professional exam
Prompt engineering	[done] Domain 4 (20%)
Tool use	[done] Domains 2 and 4
Agents	[done] Domain 1 (27%)
Production patterns	~ Depends what it covers

Take: tool use, prompt engineering, agents. **Skip the rest.**

Pay attention to: - `tool_use` with JSON schemas for reliable structured output - `tool_choice`: `auto`, `any`, forced - Handling `stop_reason` - The Message Batches API: `custom_id`, 24-hour window, 50% cheaper, no speed guarantee

---

[no] Skip - Claude with Amazon Bedrock (100-200)

[no] Skip - Claude on Google Cloud (100-200)

Section 17 of the guide lists as out of scope: **"Specific cloud provider configurations (AWS, GCP, Azure)."**

Also out of scope, and covered by these courses: rate limits, quotas, API pricing calculations, and deploying or hosting MCP servers (infrastructure, networking, containers).

These are good courses for your job. They are worth **zero points** on this exam.

[no] Skip - AI Fluency: Framework & Foundations (100)

[no] Skip - Claude 101 (100)

These are Level 100 general-use courses. This exam expects 6+ months of hands-on work with the Agent SDK, Claude Code, MCP, and the API.

Also, the ethics and safety topics in AI Fluency overlap the out-of-scope list: Constitutional AI, RLHF, and safety training methods are not tested.

Skim them if you want the vocabulary. Do not plan time for them.

The updated schedule, with courses included

The courses **replace** exercise time. They do not add to it.

Days	Study	Hands-on
1	Answer patterns, skim notes	Practice Set 1
2-4	Domain 1 notes	**Claude Code in Action** - Agent SDK + hooks - Domain 1 drill

5-6	Domain 2 notes	**Intro to MCP** - tools, resources, errors - Domain 2 drill
7-9	Domain 3 notes	**Claude Code in Action** - commands, context - Exercise 2 - Domain 3 drill
10	-	Set 2, then list your weak domains
11-13	Domain 4 notes	**Building with the Claude API** - tool use + prompt engineering only - Exercise 3 - Domain 4 drill
14-15	Domain 5 notes	Exercise 4 - Domain 5 drill
16	Scenarios	Recreate the Sample Q7 coverage failure
17	Weak domains	-
18	-	Set 3 (timed)
19	Review mistakes	-
20	Blueprint and scope lists	Rest

When a course and the guide disagree

Courses teach the product. The exam tests the guide's description of the product.

- When a course goes **deeper** than the guide (Bedrock model IDs, RAG chunking, evaluation design, streaming, prompt caching internals) - that is not tested.
- When the guide states something a course skips - **the guide wins**.

Here are things you will probably not learn from any course, but which are tested:

- `allowedTools` must include `"Task"` before a coordinator can start subagents
- The Batches API has **no speed guarantee** and no multi-turn tool calling
- **Required** schema fields make the model invent values. Make them nullable
- Strict schemas stop **syntax** errors, but not **semantic** ones
- Self-reported confidence is wrong for escalation, but acceptable for routing review work - only when calibrated with labelled data
- A bigger context window does **not** fix attention dilution

If you are unsure during a course, check the objective in `notes/domain-N-*.md`. If it is not listed there as a knowledge or skill statement, it is not on the exam.